

Chapter 2 : Processes And Process Management

Introduction

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- Examples of IPC Systems
- Communication in Client-Server Systems

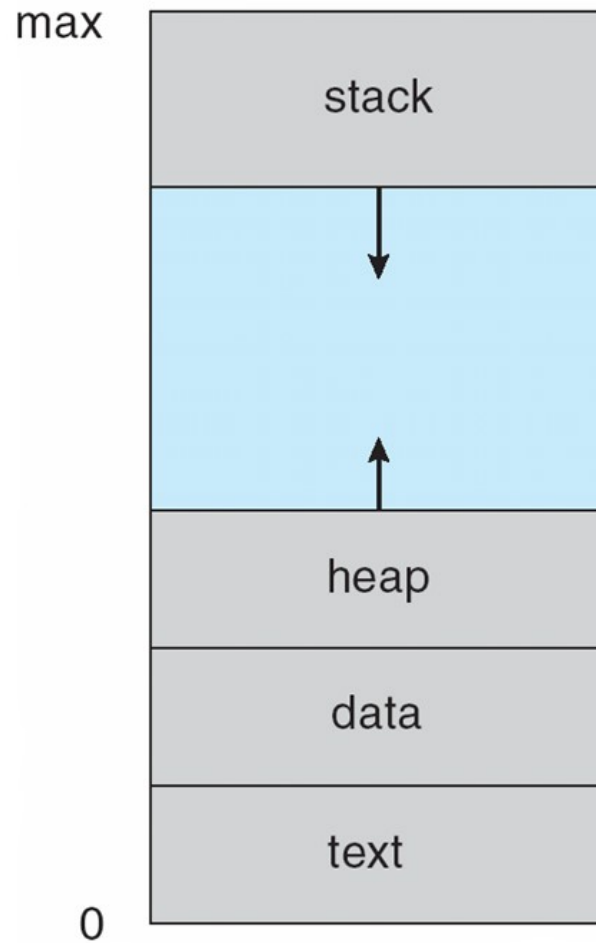
Objectives

- To introduce the notion of a process -- a program in execution, which forms the basis of all computation
- To describe the various features of processes, including scheduling, creation and termination, and communication
- To describe communication in client-server systems

Process Concept

- An operating system executes a variety of programs:
 - Batch system – jobs
 - Time-shared systems – user programs or tasks
- Textbook uses the terms *job* and *process* almost interchangeably
- Process – a program in execution; process execution must progress in sequential fashion
- A process includes:
 - program counter
 - stack
 - data section

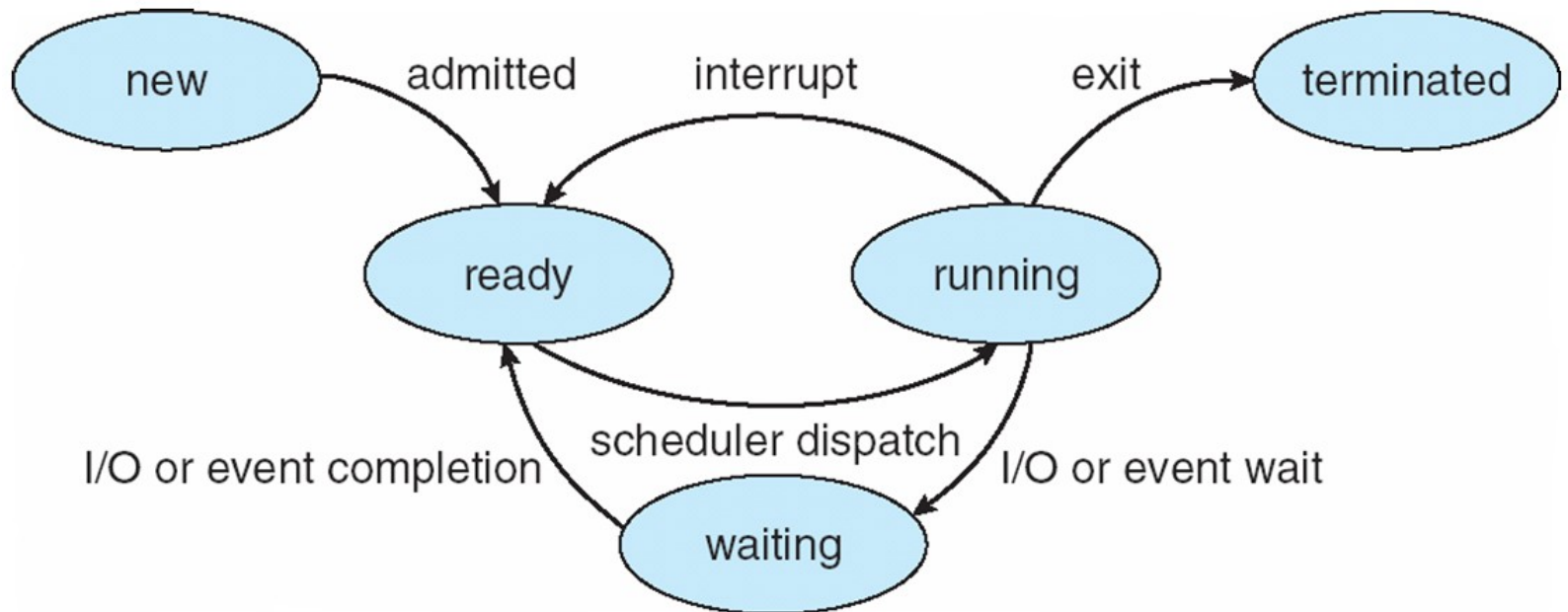
Process in Memory



Process State

- As a process executes, it changes *state*
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution

Diagram of Process State

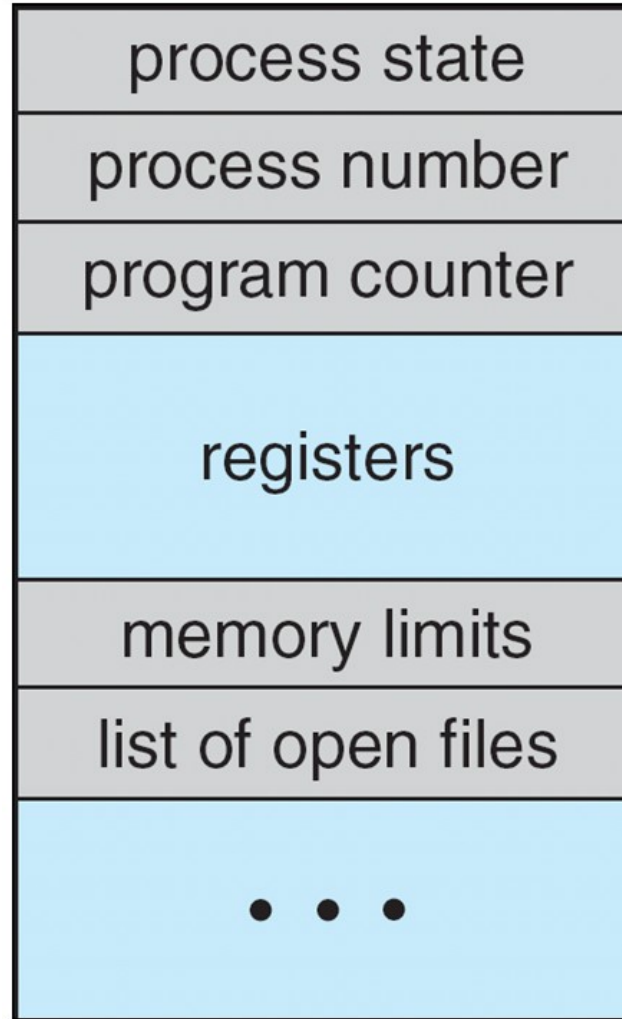


Implementation of Process- Processes Control Block (PCB)

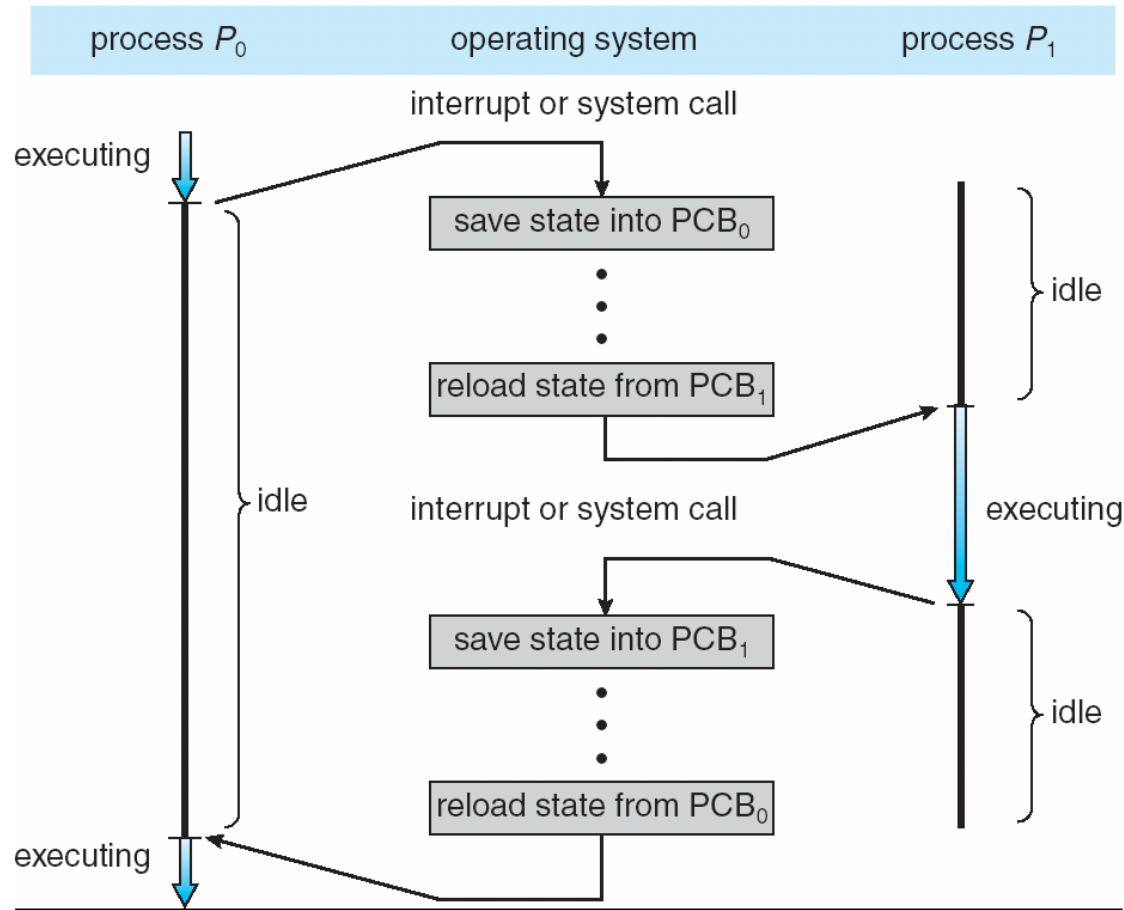
Information associated with each process

- Process state
- Program counter
- CPU registers
- CPU scheduling information
- Memory-management information
- Accounting information
- I/O status information

Process Control Block (PCB)

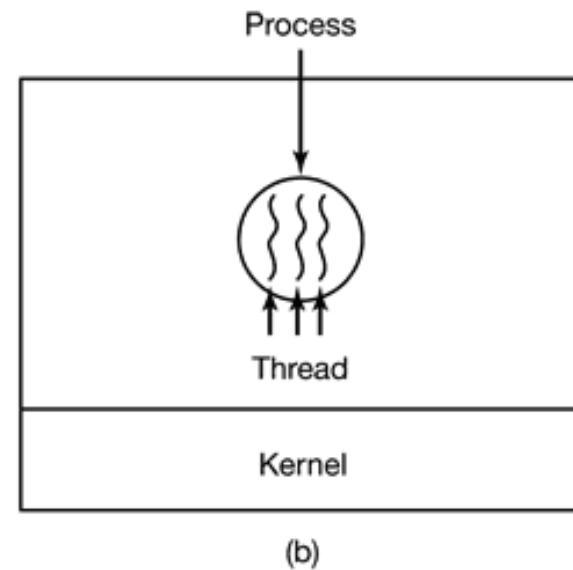
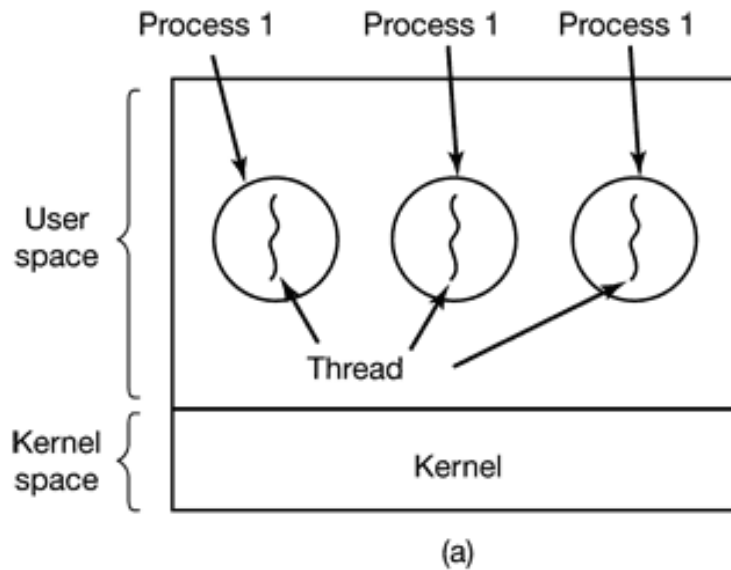


CPU Switch From Process to Process



The Thread Model

- Lightweight processes



Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

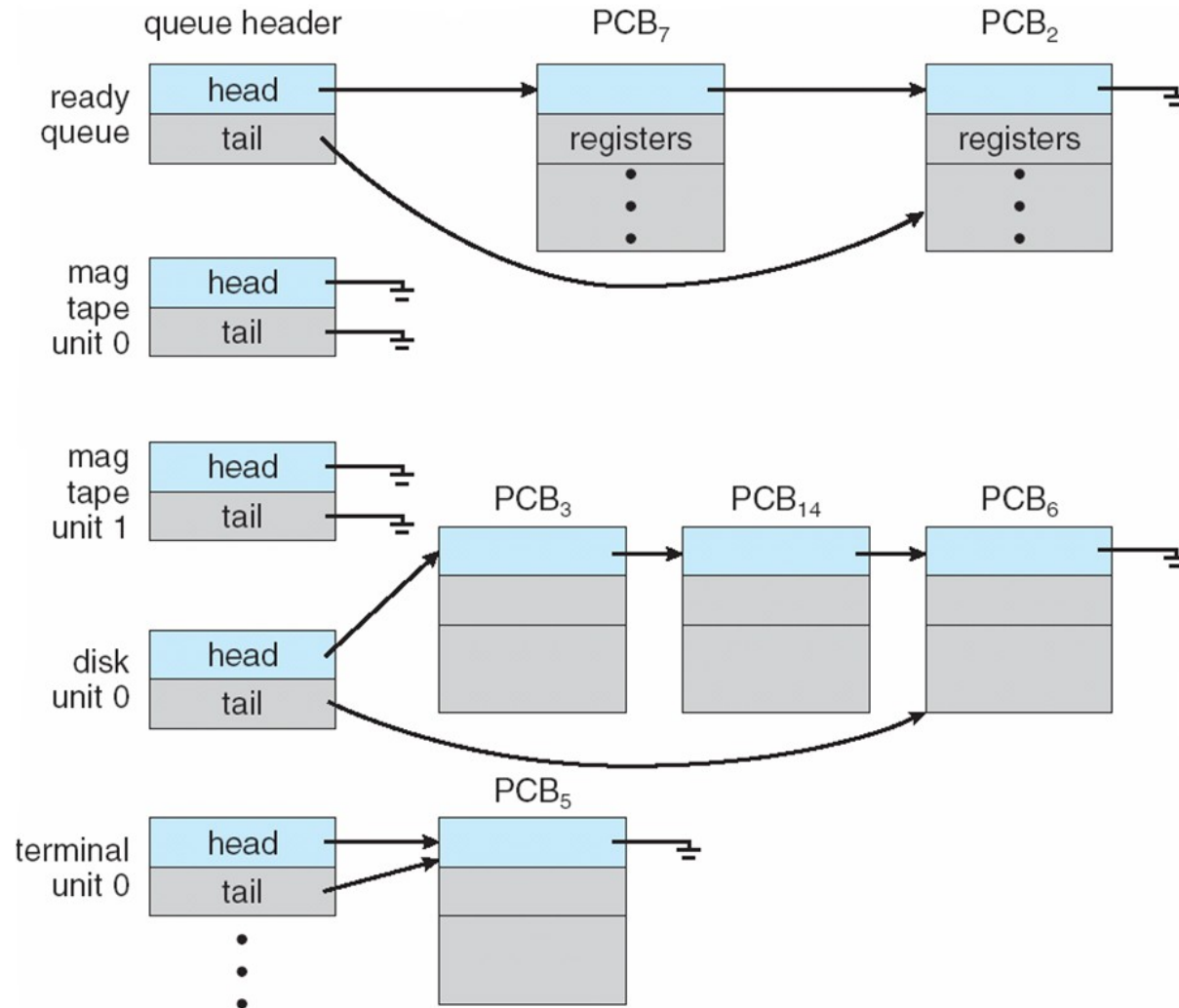
Why Threads?

- **Quasi Parallelism**
- **They are easier to create and destroy than processes**

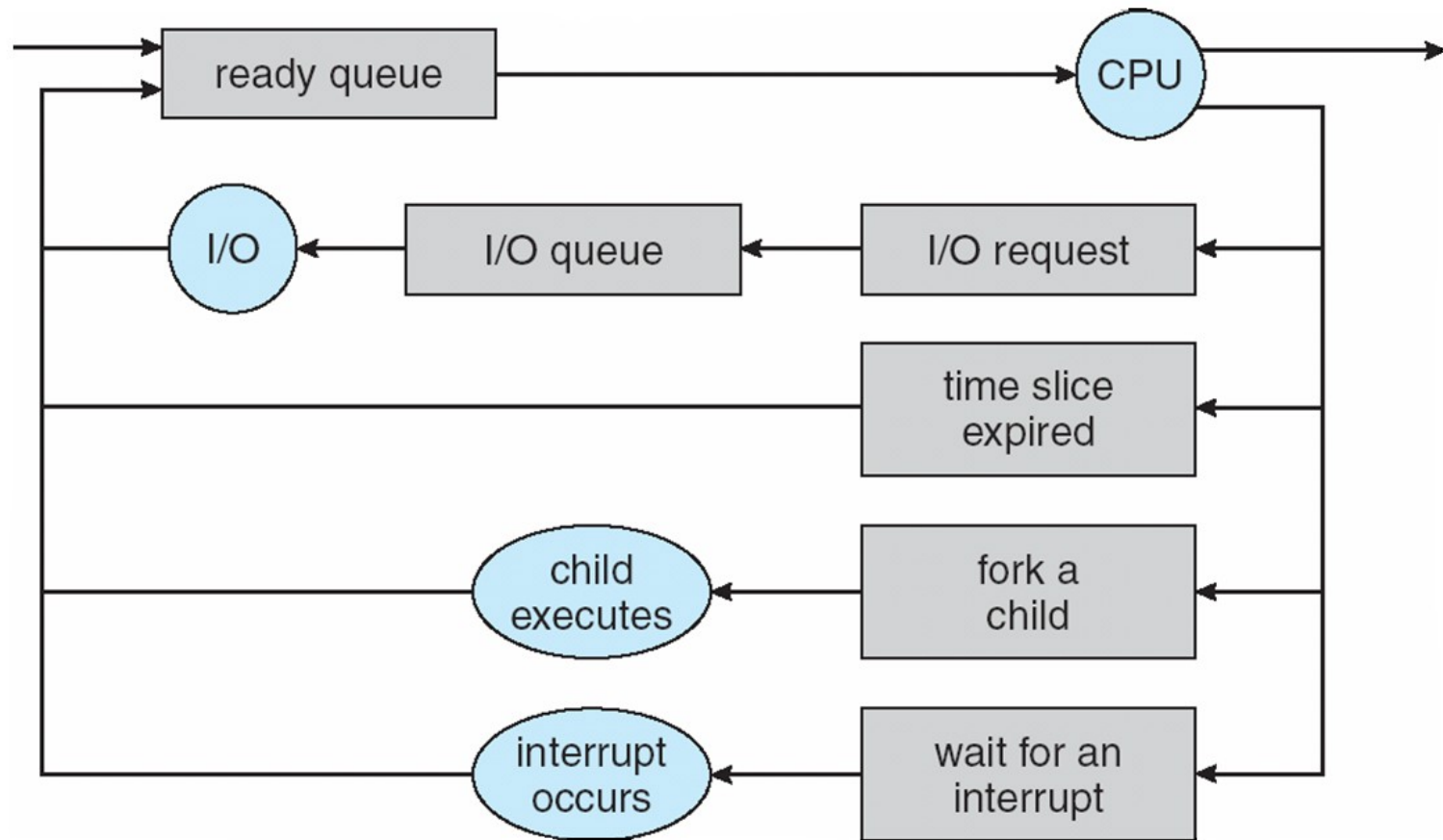
Process Scheduling Queues

- **Job queue** – set of all processes in the system
- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
- **Device queues** – set of processes waiting for an I/O device
- Processes migrate among the various queues

Ready Queue And Various I/O Device Queues



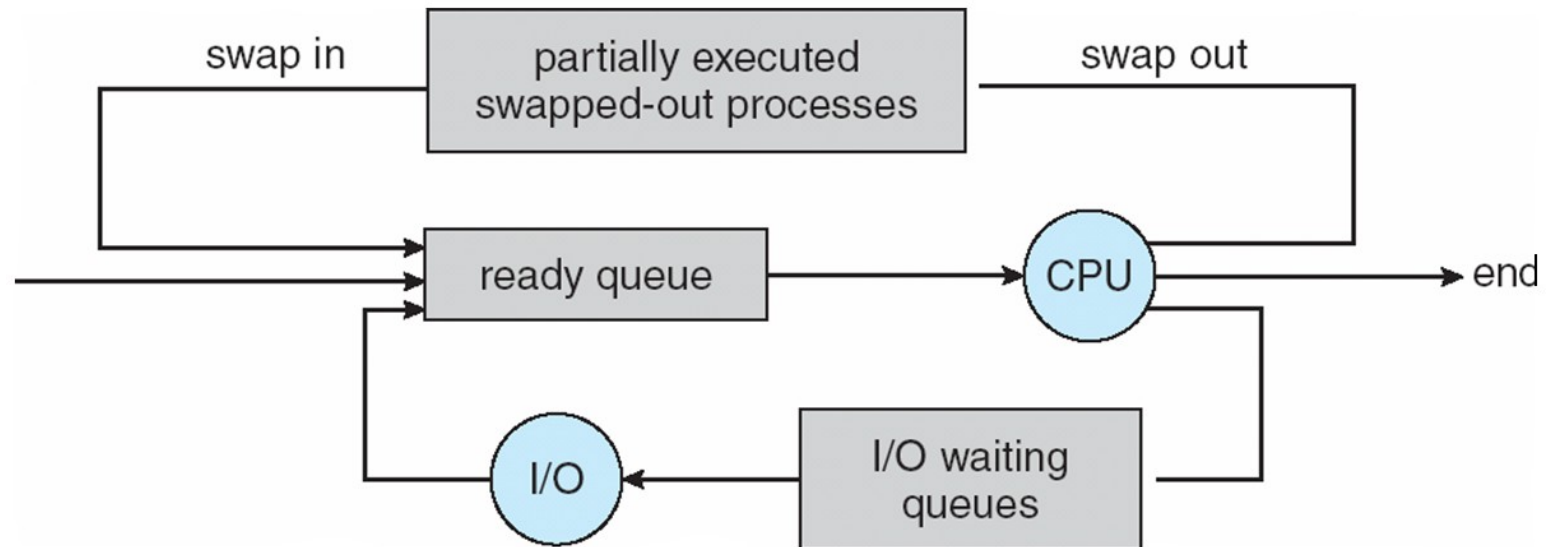
Representation of Process Scheduling



Schedulers

- Selects which processes should be brought into the ready queue
- Selects which process should be executed next and allocates CPU

Addition of Medium Term Scheduling



Schedulers (Cont)

- Short-term scheduler is invoked very frequently (milliseconds) $\Rightarrow$ (must be fast)
- Long-term scheduler is invoked very infrequently (seconds, minutes) $\Rightarrow$ (may be slow)
- The long-term scheduler controls the *degree of multiprogramming*
- Processes can be described as either:
 - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
 - **CPU-bound process** – spends more time doing computations; few very long CPU bursts

Context Switch

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
- Time dependent on hardware support

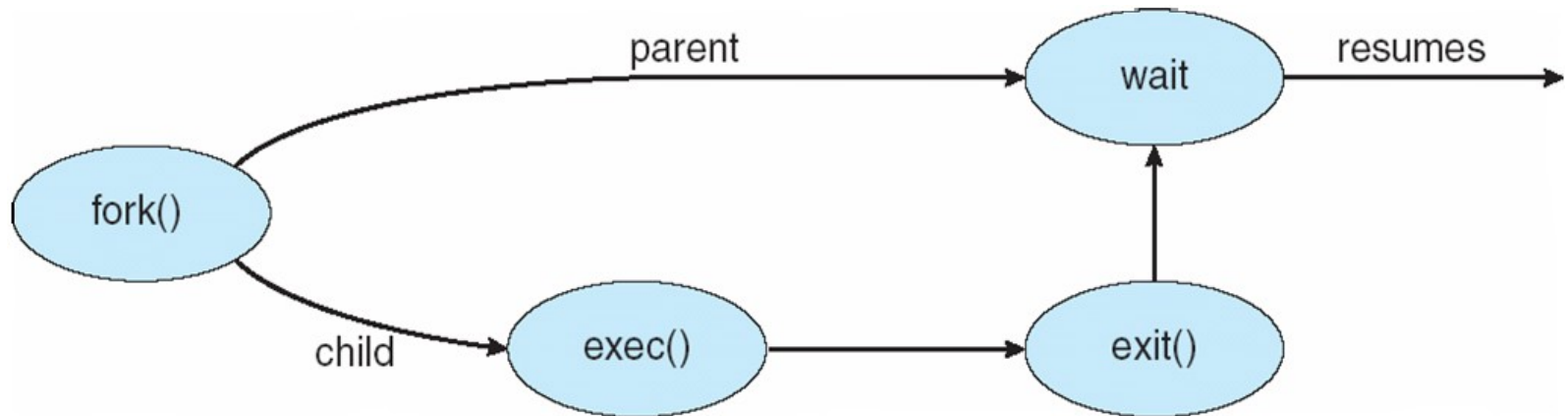
Process Creation

- Process creation can be triggered in either of
 - System initialization.
 - Execution of a process creation system call by a running process.
 - A user request to create a new process.
 - Initiation of a batch job.
- **Parent** process create **children** processes, which, in turn create other processes, forming a tree of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution
 - Parent and children execute concurrently
 - Parent waits until children terminate

Process Creation (Cont)

- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - **fork** system call creates new process
 - **exec** system call used after a **fork** to replace the process' memory space with a new program

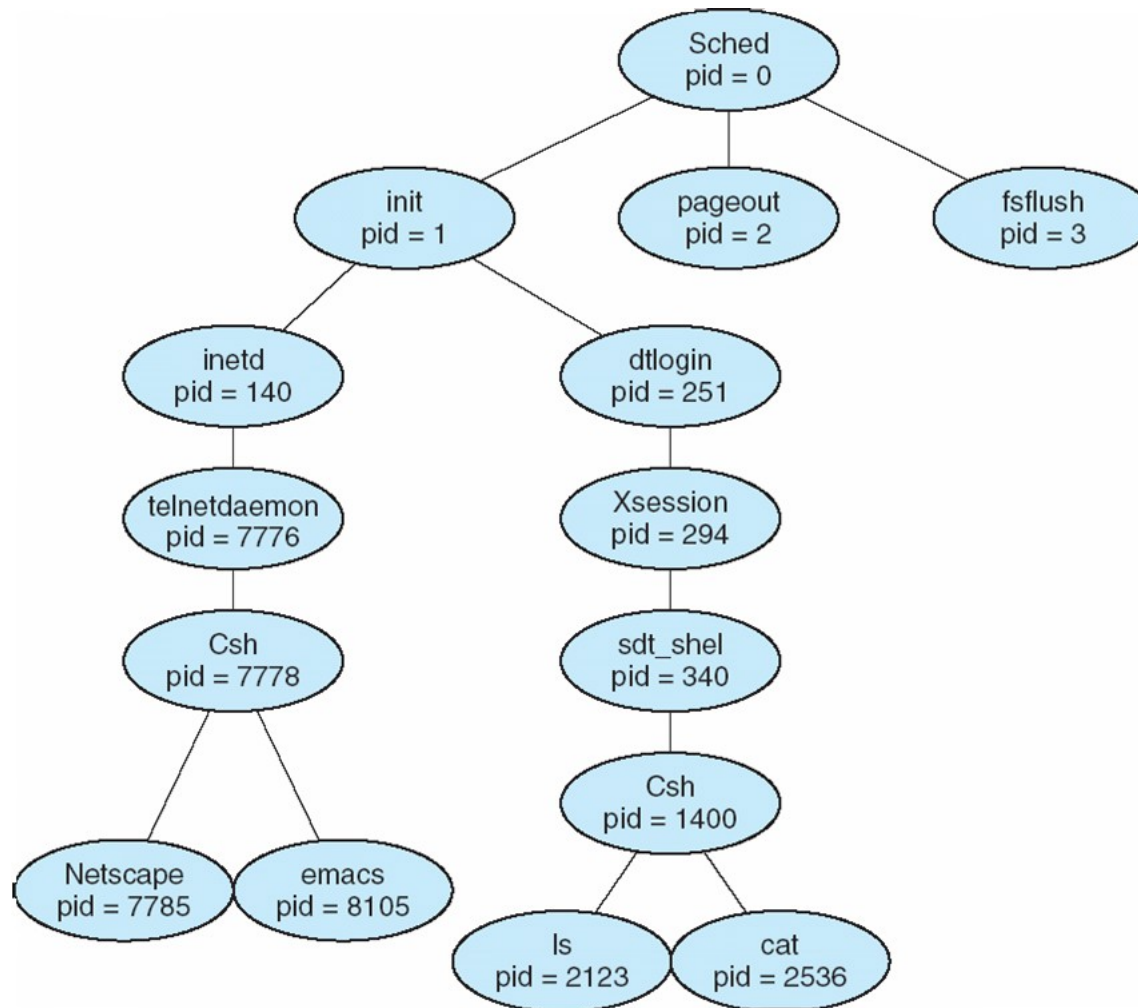
Process Creation



C Program Forking Separate Process

```
int main()
{
    pid_t  pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child
        to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```

A tree of processes on a typical Solaris



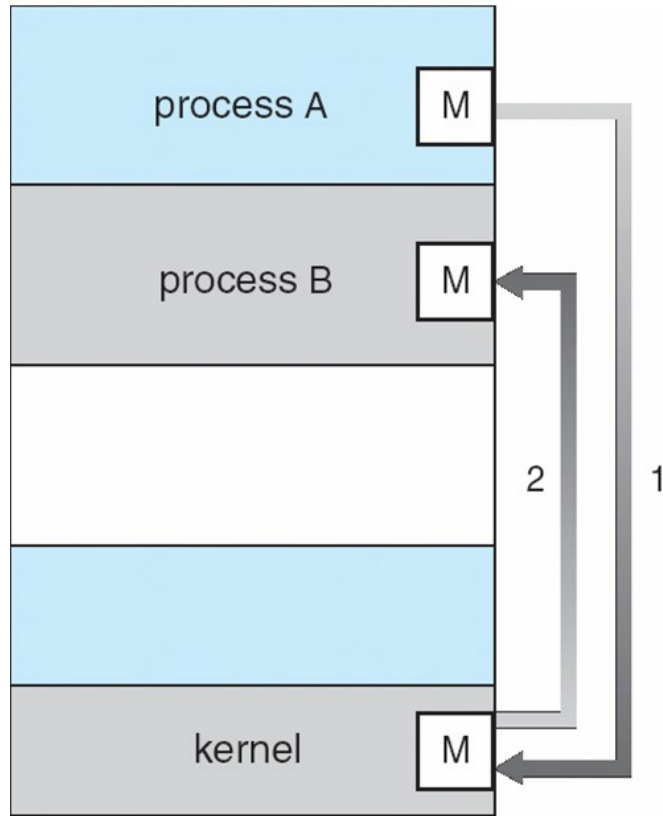
Process Termination

- ***Process Termination***
 - Normal exit (voluntary).
 - Error exit (voluntary).
 - Fatal error (involuntary).
 - Killed by another process (involuntary)
- Process executes last statement and asks the operating system to delete it (**exit**)
 - Output data from child to parent (via **wait**)
 - Process' resources are de-allocated by operating system
- Parent may terminate execution of children processes (**abort**)
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - If parent is exiting
 - Some operating system do not allow child to continue if its parent terminates
 - All children terminated - **cascading termination**

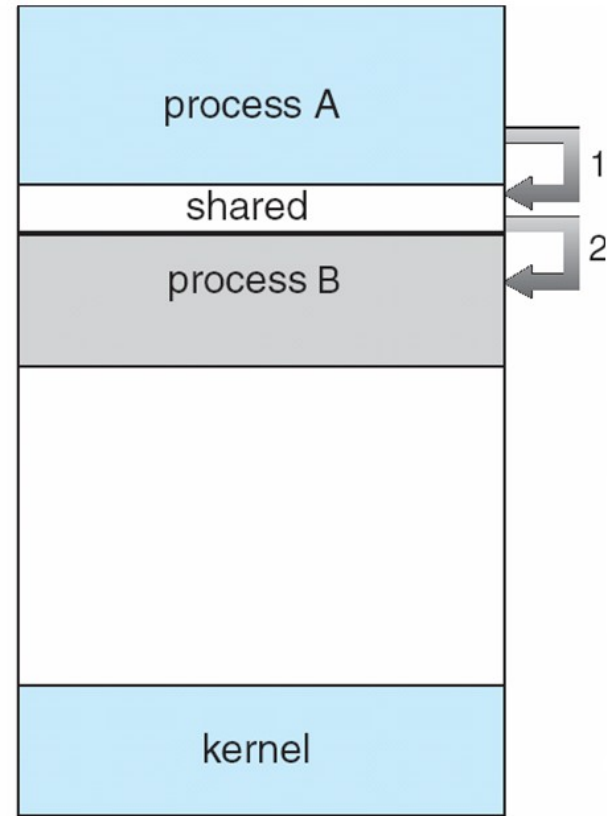
Interprocess Communication

- Processes within a system may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - Shared memory
 - Message passing

Communications Models



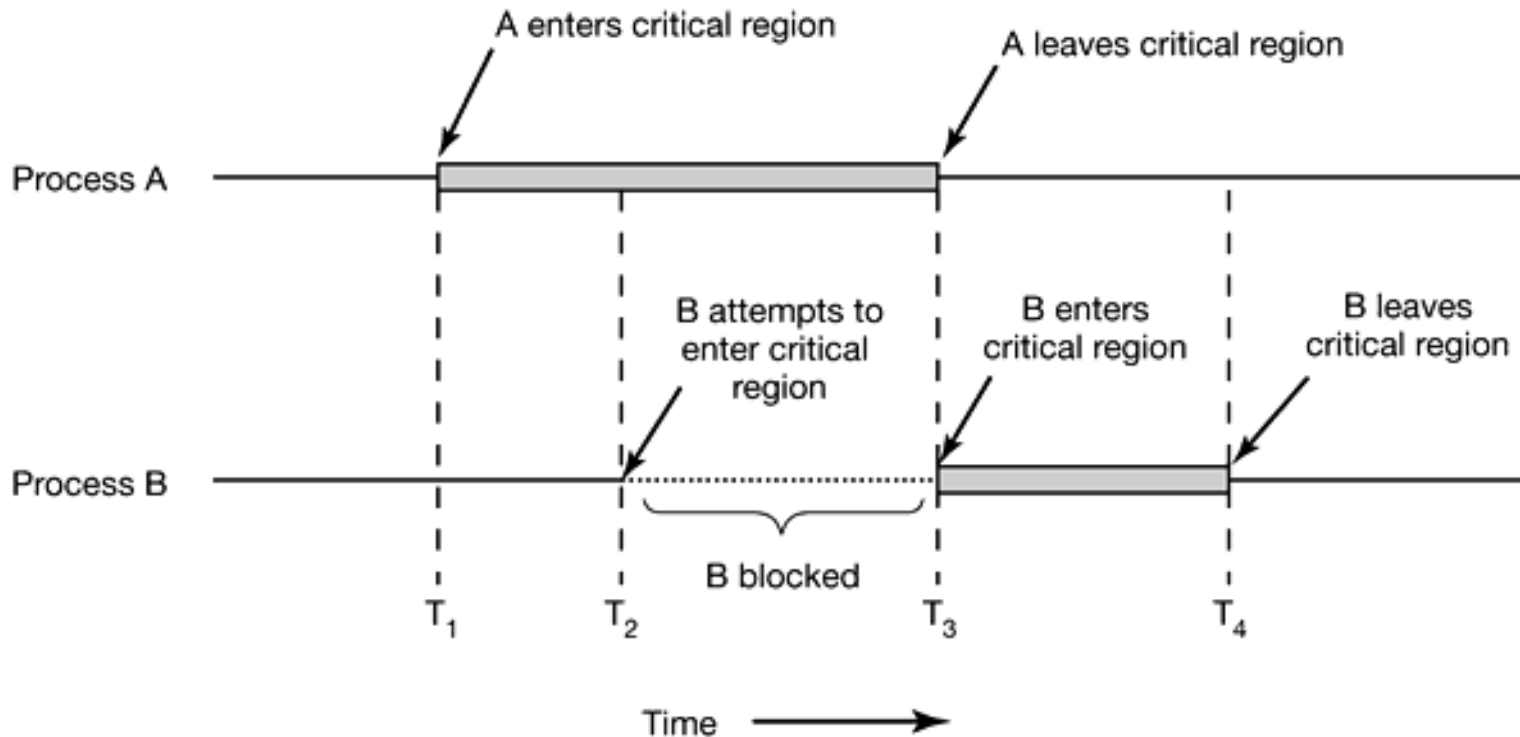
(a)



(b)

Communications Model...

Race Conditions , Critical sections, & Mutual Exclusion



Communication Models...

- Although mutual exclusion avoids race conditions, it is not sufficient for having parallel processes cooperate correctly and efficiently using shared data.
- There needs to be four conditions to hold to have a good solution:
 1. No two processes may be simultaneously inside their critical regions
 2. No assumptions may be made about speeds or the number of CPUs
 3. No process running outside its critical region may block other processes
 4. No process should have to wait forever to enter its critical region

Communication Models...

- Techniques for achieving mutual exclusion
 - Mutual Exclusion with Busy Waiting
 - Disabling Interrupts
 - Lock Variables
 - Strict Alternation
 - Peterson's Algorithm
 - The TSL Instruction

Communication Models...

Assignment- Group 1

- Prepare a comparative analysis of the busy-waiting algorithms described in the previous slide.

Communication Models...

- **Mutual Exclusion without Busy Waiting**
 - Problems of Busy-Waiting
 - Wastes CPU time
 - Priority inversion problem
- Blocking instead of wasting CPU time- Without Busy Waiting

Cooperating Processes

- **Independent** process cannot affect or be affected by the execution of another process
- **Cooperating** process can affect or be affected by the execution of another process
- Advantages of process cooperation
 - Information sharing
 - Computation speed-up
 - Modularity
 - Convenience

Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
 - *unbounded-buffer* places no practical limit on the size of the buffer
 - *bounded-buffer* assumes that there is a fixed buffer size

Bounded-Buffer – Producer

```
while (true) {  
    /* Produce an item */  
    while (((in = (in + 1) % BUFFER SIZE  
count) == out)  
        ; /* do nothing -- no free buffers */  
    buffer[in] = item;  
    in = (in + 1) % BUFFER SIZE;  
}
```

Bounded Buffer – Consumer

```
while (true) {  
    while (in == out)  
        ; // do nothing  
    -- nothing to consume  
  
    // remove an item from the  
    buffer  
    item = buffer[out];  
    out = (out + 1) % BUFFER  
    SIZE;  
    return item;  
}
```

Producer-Consumer Problem...

- **Sleep and Wakeup** – uses two system calls called sleep and wakeup. Sleep causes the caller to block, that is, be suspended until another process wakes it up. The wakeup call awakens a process blocked by sleep.

Producer-Consumer Problem...

```
#define N 100    /* number of slots in the buffer
*/
int count = 0;   /* number of items in the buffer
*/

void producer (void)
{
    int item;

    while (TRUE) {          /* repeat forever */
        item = produce_item(); /* generate next
item */
        if (count == N) sleep(); /* if buffer is full, go
to sleep */
        insert_item(item);    /* put item in buffer */
        count = count + 1;    /* increment count of
items in buffer */
        if (count == 1) wakeup(consumer); /* was
buffer empty? */
    }
}
```

```
void consumer(void)
{
    int item;

    while (TRUE) {          /* repeat
forever */
        if (count == 0) sleep(); /* if
buffer is empty, got to sleep */
        item = remove_item(); /*
take item out of buffer */
        count = count - 1;    /*
decrement count of items in buffer
*/
        if (count == N - 1)
            wakeup(producer); /* was buffer
full? */
        consume_item(item);    /*
print item */
    }
}
```

Semaphores

- Uses an integer variable (semaphore) to count the number of wakeups saved for future use.
- A semaphore could have the value 0, indicating that no wakeups were saved, or some positive value if one or more wakeups were pending.
- Two operations, down and up (generalizations of sleep and wakeup, respectively) would be performed on this variable.
- The down operation on a semaphore checks to see if the value is greater than 0.
 - *If so, it decrements the value (i.e., uses up one stored wakeup) and just continues.*
 - *If the value is 0, the process is put to sleep without completing the down for the moment. Checking the value, changing it and possibly going to sleep, is all done as a single, indivisible **atomic action**.*

Semaphores...

- *This atomicity is absolutely essential to solving synchronization problems and avoiding race conditions.*
- The up operation increments the value of the semaphore addressed.
 - If one or more processes were sleeping on that semaphore, unable to complete an earlier down operation, one of them is chosen by the system (e.g., at random) and is allowed to complete its down.

Other Possible IPC Mechanisms

- **Classical IPC Problems**
 - *The Dining Philosophers Problem*
 - *The Readers and Writers Problem*
 - *The Sleeping Barber Problem*
- **Assignment Group II-**
 - Semaphore
 - Monitors
 - Message Passing

Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - **send**(*message*) – message size fixed or variable
 - **receive**(*message*)
- If P and Q wish to communicate, they need to:
 - establish a *communication link* between them
 - exchange messages via send/receive
- Implementation of communication link

Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)

SCHEDULING

- Why Scheduling?
 - Multiprogramming
- The part of the operating system that makes the choice is called the **scheduler** and the algorithm it uses is called the **scheduling algorithm**.
- Process Behavior:
 - Compute-Bound
 - I/O-Bound
- When to Schedule?
 - When a new process is created
 - When a process exits.
 - When a process blocks on I/O, on a semaphore, or for some other reason
 - When an I/O interrupt occurs

SCHEDULING...

- Scheduling algorithms can be divided into two categories:
 - Non-preemptive- run through completion
 - Suited for batch systems
 - Preemptive- the scheduler picks a process and lets it run for a maximum of some fixed time.
 - Suited for interactive systems

SCHEDULING...

- *Scheduling Algorithm Goals*
 - All systems
 - Fairness - giving each process a fair share of the CPU
 - Policy enforcement - seeing that stated policy is carried out
 - Balance - keeping all parts of the system busy
 - Batch systems
 - Throughput - maximize jobs per hour
 - Turnaround time - minimize time between submission and termination
 - CPU utilization - keep the CPU busy all the time
 - Interactive systems
 - Response time - respond to requests quickly
 - Proportionality - meet users' expectations
 - Real-time systems
 - Meeting deadlines - avoid losing data
 - Predictability - avoid quality degradation in multimedia systems

Scheduling in Batch Systems

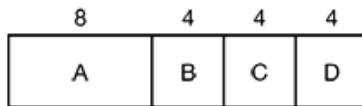
- **First-Come First-Served** —
 - It is a nonpreemptive algorithm, processes are assigned the CPU in the order they request it.
 - Basically, there is a single queue of ready processes.
 - When the first job enters the system, it is started immediately and allowed to run as long as it wants to.
- ***Strength***
 - Easy to understand and equally easy to program
- ***Limitations***
 - Performs better for long processes and it favors I/O bound processes.

Process	Arrival time	Service time (sec)	Start time	Finish time	Turnaround time
P1	0	1	0	1	1
P2	1	100	1	101	100
P3	2	1	101	102	100
P4	3	100	102	202	199

Scheduling in Batch Systems...

- **Shortest Job First –**

- is a nonpreemptive batch algorithm, that assumes the run times are known in advance, and picks the shortest job first.
- When all the jobs are available simultaneously, shortest job first is optimal.
- Consider the following four jobs A, B, C, and D with the indicated run times.



(a)



(b)

Turnaround time using FCFS (a)	
A	8 minutes
B	12 minutes
C	16 minutes
D	20 minutes
Average	14 minutes

Turnaround time using SJF (b)

B	4 minutes
C	8 minutes
D	12 minutes
A	20 minutes
Average	11 minutes

Scheduling in Batch Systems...

- **Shortest Remaining Time Next**
 - is a preemptive version of shortest job first. With this algorithm, the scheduler always chooses the process whose remaining run time is the shortest. Again here, the run time has to be known in advance.
 - When a new job arrives, its total time is compared to the current process' remaining time.
 - If the new job needs less time to finish than the current process, the current process is suspended and the new job started. This scheme allows new short jobs to get good service.

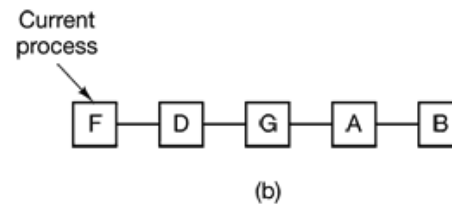
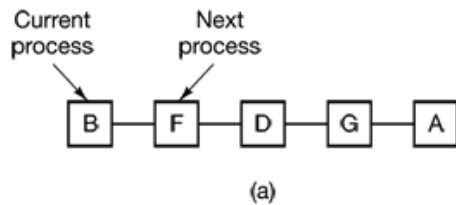
Scheduling in Interactive Systems

- **Round-Robin Scheduling**

- each process is assigned a time interval, called its **quantum**,
- If the process is still running at the end of the quantum, the CPU is preempted and given to another process.
- If the process has blocked or finished before the quantum has elapsed, the CPU is switched to another process.
- the length of the quantum should be set carefully:
 - setting the quantum too short causes too many process switches and lowers the CPU efficiency, and
 - setting it too long may cause poor response to short interactive requests.
 - A quantum around 20-50 milliseconds is often a reasonable compromise.

Scheduling in Interactive Systems...

- Strength of Round-Robin algorithm
 - Simple to implement
 - Fair



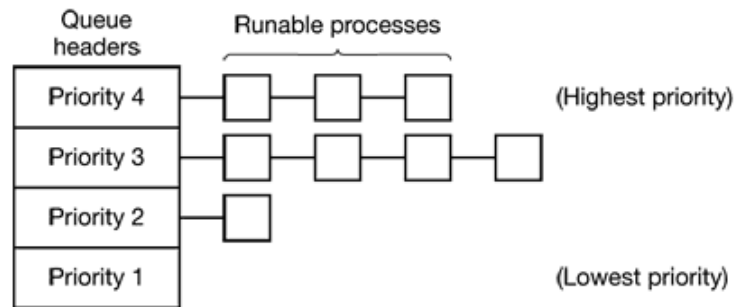
Scheduling in Interactive Systems...

- **Priority Scheduling**

- each process is assigned a priority, and the runnable process with the highest priority is allowed to run.
- To prevent high-priority processes from running indefinitely, the scheduler may decrease the priority of the currently running process at each clock tick (i.e., at each clock interrupt).
- If this action causes its priority to drop below that of the next highest process, a process switch occurs.
- It is often convenient to group processes into priority classes and use priority scheduling among the classes but round-robin scheduling within each class.

Scheduling in Interactive Systems...

- Priority Scheduling



Scheduling in Interactive Systems...

- **Shortest Process Next:**

- the shortest process will be run first
- **Challenge:** figuring out which of the currently runnable processes is the shortest one
- A common approach to deal with this is to approximate its value based on past behavior.
- The next CPU burst is generally predicted as *an exponential average of the measured lengths of previous CPU bursts*.
- Let t_n be the length of the n^{th} CPU (which is the most recent CPU burst) and let T_{n+1} be our predicted value for the next CPU burst.



Scheduling in Interactive Systems...

- **Shortest Process Next:**

- Then, for $\alpha \leq 0 \leq 1$, an exponential average can be defined as
 - $T_{n+1} = \alpha t_n + (1 - \alpha)T_n$;
 - where t_n contains our most recent information
and T_n stores the past history.
- The parameter α controls the relative weight of recent and past history in our prediction.
- If $\alpha = 0$, then $T_{n+1} = T_n$ and recent history has no effect
- If $\alpha = 1$, then $T_{n+1} = t_n$ and only the most recent CPU burst matters
- But more commonly, $\alpha = 1/2$, so recent history and past history are equally weighted.

Scheduling in Interactive Systems...

- **Lottery Scheduling:**

- processes are given lottery tickets for various system resources, such as CPU time and whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the resource.

- **Fair-Share Scheduling:**

- takes into account the user that owns a process before scheduling it.
 - In this model, a user is allocated some fraction of the CPU and the scheduler picks processes in such a way as to enforce it.
 - User 1 has four processes, *A*, *B*, *C*, and *D*, and user 2 has only 1 process, *E*. If round-robin scheduling is used, a possible scheduling sequence that meets all the constraints is : *A E B E C E D E A E B E C E D E ...*

Scheduling in Real-Time Systems

- A *real-time* system is one in which time plays an essential role.
- Deadline-based scheduling
- Real-time systems are generally categorized as:
 - *Hard real time* – systems where absolute deadlines that must be met
 - *Soft real time*- systems where missing an occasional deadline is undesirable, but nevertheless tolerable.
- Real-time scheduling algorithms can be *static* or *dynamic*.
 - Static algorithms assign each process a fixed priority in advance and then do prioritized preemptive scheduling using those priorities.
 - works when there is perfect information available in advance about the work needed to be done and the deadlines that have to be met.

Scheduling in Real-Time Systems...

- **Rate Monotonic Scheduling** (Assignment group III)

Deadlock

- Computer systems are full of resources that can only be used by one process at a time.
- Common examples include printers, tape drives, and slots in the system's internal tables.
- Having two processes simultaneously writing to the printer leads to gibberish.
- Having two processes using the same file system table slot will invariably lead to a corrupted file system.
- Consequently, all operating systems have the ability to (temporarily) grant a process exclusive access to certain resources.

Deadlock...

- For many applications, a process needs exclusive access to not one resource, but several.
 - Suppose, for example, two processes (*A* and *B*) each want to record a scanned document on a CD. The following can happen.
 - Process *A* requests permission to use the scanner and is granted it.
 - Process *B* is programmed differently and requests the CD recorder first and is also granted it.
 - Now *A* asks for the CD recorder, but the request is denied until *B* releases it.
 - Instead of releasing the CD recorder, *B* asks for the scanner.
 - At this point both processes are blocked and will remain so forever. This situation is called a **deadlock**.

Deadlock...

- Deadlock can be defined formally as follows:
 - *A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.* Because all the processes are waiting, none of them will ever cause any of the events that could wake up any of the other members of the set, and all the processes continue to wait forever.
- **Resources**
 - Deadlocks can occur when processes have been granted exclusive access to devices, files, and so forth.
 - Resources come in two types: preemptable and nonpreemptable.
 - A *preemptable* resource is one that can be taken away from the process owning it with no ill effects. Memory is an example of a preemptable resource.
 - A *nonpreemptable resource*, in contrast, is one that cannot be taken away from its current owner without causing the computation to fail.
 - If a process has begun to burn a CD-ROM, suddenly taking the CD recorder away from it and giving it to another process will result in a garbled CD, CD recorders are not preemptable at an arbitrary moment.

Deadlock...

- *Conditions for Deadlock*
 - All of the following four conditions must hold for there to be a deadlock:
 - Mutual exclusion condition. Each resource is either currently assigned to exactly one process or is available.
 - Hold and wait condition. Processes currently holding resources granted earlier can request new resources.
 - No preemption condition. Resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them.
 - Circular wait condition. There must be a circular chain of two or more processes, each of which is waiting for a resource held by the next member of the chain.

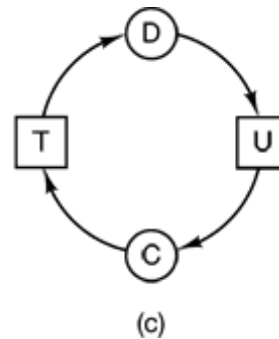
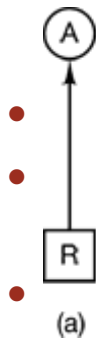
Deadlock...

- **Starvation**
 - A problem closely related to deadlock is **starvation**. In a dynamic system, requests for resources happen all the time. Some policy is needed to make a decision about who gets which resource when.
 - This policy, although seemingly reasonable, may lead to some processes never getting service even though they are not deadlocked.

Deadlock...

- **Deadlock Modeling**

- Deadlock can be modeled using directed graphs.
- The graphs have two kinds of nodes: processes, shown as circles, and resources, shown as squares.
- An arc from a resource node (square) to a process node (circle) means that the resource has previously been requested by, granted to, and is currently held by that process.



assigned to process A
process is currently blocked waiting
cess B is waiting for resource S.

Deadlock...

- *Dealing with Deadlocks*

- In general, four strategies are used for dealing with deadlocks. These are:
 - Just ignoring the problem altogether (the Ostrich Algorithm)
 - Detection and recovery (let deadlocks occur, detect them, and take action.)
 - Some of the techniques to recover from deadlock are
 - **Recovery through Preemption - temporarily**
 - **Recovery through Rollback**
 - **Recovery through Killing Processes**
 - Dynamic avoidance by careful resource allocation.
 - Prevention, by structurally negating one of the four conditions necessary to cause a deadlock.